Name (LN, FN):	KHAN, Sarah & QUIZON, Marc	Date:	**2/27/2026**
Section:	S18	Grade:	

9.0 HTTP Service

Hypertext Transfer Protocol (HTTP) is an application protocol that defines how messages are formatted and transmitted, and the actions Web servers and browsers should take in response to commands. HTTP is widely used in the World Wide Web for communication between web browsers and web servers. The Linux system can be configured as a web server by installing the Apache web server. Note that Apache web server is **not** the only web server that can be installed on the Linux system. The normal port number for the HTTP server is 80.

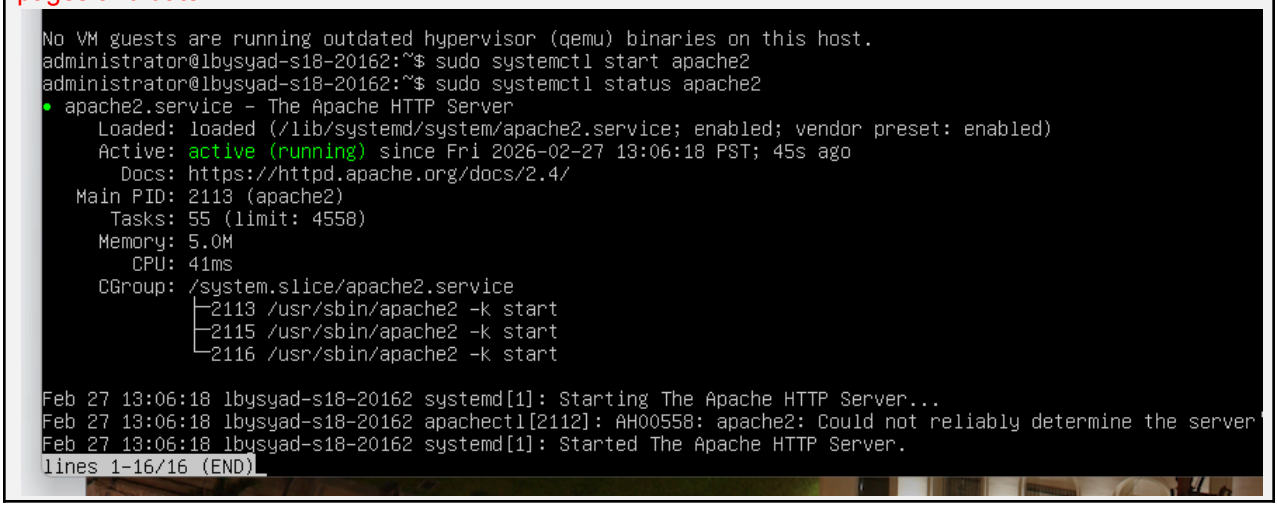
For this laboratory hands-on the server is in the Proxmox virtual environment. Since the virtual machines in the Proxmox server share the same IP address, port redirection is used to direct a service to a virtual machine. The setup in this Proxmox server use port number xxxxx to redirect the HTTP service (which is port 80) to your assigned virtual machine.

9.1. Installing and Configuring Apache Web Server

Step 1 Login to the `administrator` account of your virtual machine.

1a. On the command prompt, type in the command '`sudo apt install apache2`'. Select 'yes' when asked if you want to install it. Wait for it to finish installing. Start the web server by using the command '`sudo systemctl start apache2`'. Verify that it is running by using '`sudo systemctl status apache2`'. What is the purpose of the `apache2` package?

Apache2 is used to receive requests from clients and send back their requests which may include web pages and date



```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
administrator@lbysyad-s18-20162:~$ sudo systemctl start apache2
administrator@lbysyad-s18-20162:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2026-02-27 13:06:18 PST; 45s ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2113 (apache2)
      Tasks: 55 (limit: 4558)
     Memory: 5.0M
        CPU: 41ms
     CGroup: /system.slice/apache2.service
             ├─2113 /usr/sbin/apache2 -k start
             ├─2115 /usr/sbin/apache2 -k start
             └─2116 /usr/sbin/apache2 -k start

Feb 27 13:06:18 lbysyad-s18-20162 systemd[1]: Starting The Apache HTTP Server...
Feb 27 13:06:18 lbysyad-s18-20162 apachectl[2112]: AH00558: apache2: Could not reliably determine the server'
Feb 27 13:06:18 lbysyad-s18-20162 systemd[1]: Started The Apache HTTP Server.
lines 1-16/16 (END)
```

Step 2 Using a browser on your physical machine, access your now web server with the following as the URL: **http**://ccscloud.dlsu.edu.ph:xxxxx where xxxxx is the assigned port number to your group for forwarding HTTP traffic to your virtual machine. Ask your instructor if you are not yet assigned a port forwarding number.

2a. Briefly describe the contents of the web page that you see on your browser.

The output is Ubuntu Default Welcome Page used to test the correct operation of the Apache2 server.

Step 3 Go back to the Linux system and at the prompt change the directory to `/etc/apache2/sites-available`.

3a. Open the file `000-default.conf` and look for the section called `DocumentRoot`.

3b. What is the directory entry following the section `DocumentRoot`?

```
<VirtualHost *:80>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.
        #ServerName www.example.com

        ServerAdmin webmaster@localhost
        DocumentRoot /var/www/html

        # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
        # error, crit, alert, emerg.
        # It is also possible to configure the loglevel for particular
        # modules, e.g.
        #LogLevel info ssl:warn

        ErrorLog ${APACHE_LOG_DIR}/error.log
        CustomLog ${APACHE_LOG_DIR}/access.log combined

        # For most configuration files from conf-available/, which are
        # enabled or disabled at a global level, it is possible to
        # include a line for only one particular virtual host. For example the
        # following line enables the CGI configuration for this host only
        # after it has been globally disabled with "a2disconf".
        #Include conf-available/serve-cgi-bin.conf
</VirtualHost>

# vim: syntax=apache ts=4 sw=4 sts=4 sr noet
```

/var/www/html

Step 4 Go to the directory indicated in the `DocumentRoot` by typing `cd '/var/www/html'`.

4a. Rename the `index.html` to `index.html.orig` by typing in '`sudo mv index.html index.html.orig`'.

4b. Go back to your browser, clear the cache and try to access the web page (**http**://ccscloud.dlsu.edu.ph:xxxxx) again.

4c. Did you see the same webpage as from Step 2? Why or why not?

No, instead it leads us to the index of /var/www/html. It shows that it contains index.html.orig, which is the file we renamed earlier..

4d. Is there a web page rendered by the browser? Does it resemble a directory in your server?

Yes, it shows a plain list of files which is the index.html.orig file. Since we renamed the file, the system is unable to find index.html. So, it went to 'Directory Listing'.

Step 5 On the Linux system, go to the directory `/etc/apache2` and edit the file `apache2.conf`.

5a. Look for the section `<Directory /var/www/>` and edit the line `Options Indexes FollowSymLinks` to `Options FollowSymLinks`. What are the `Indexes` and `FollowSymLinks` options for?

The 'indexes' allows the server to generate and display a formatted listing of the files within a directory if no default index file, and the 'FollowSymLinks' allows the web server to follow symbolic links within the directory.

5b. After editing and saving the `apache2.conf` file, go to the directory `/var/www/html` and create a file `.htaccess`. What is the purpose of this file?

The '.htaccess' file is used for directory specific configurations wherein the user doesn't have to use root or sudo to access the main system files.

5c. Edit the file `.htaccess` to contain a line '`Options -Indexes`'. Make sure that the file ends with a blank line.

5d. After editing the `.htaccess` file, restart the Apache web server on the Linux system. This can be done by typing the command `sudo systemctl restart apache2`.

5e. Clear the browser cache and reload the web page.

5f. Were you able to see the directories as from Step 4d? How did '`Options -Indexes`' inside the `.htaccess` file affected the output of the web server?

Forbidden

You don't have permission to access this resource.

Apache/2.4.52 (Ubuntu) Server at ccscloud.dlsu.edu.ph Port 62063

No. The website is now forbidden to access. Adding the 'Options -Indexes' made the website forbidden for the user to access when the index.html file is not found. This acts as a safety precaution.

Step 6 Go back to the Linux system and create an index.html file in the directory `/var/www/html`. This can be done by using the command `sudo touch index.html`.

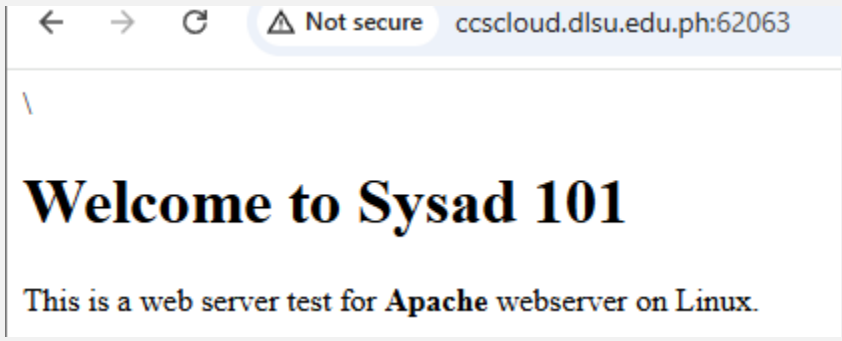
6a. Edit the created file `index.html` to contain the following:

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>Welcome to Sysad 101</h1>
<p>This is a web server test for <b> Apache </b> web server on Linux.</p>

</body>
</html>
```

6b. Clear the browser cache and reload the web page.

6c. Did the web page change?

Yes.



6d. If your answer to 6c is yes, describe the contents of the web page. If your answer is no, is the web page the same as Step 2a, 4d, or 5f?

It opened a web page with the headline 'Welcome to Sysad 101" and a caption showing that it is a web server test for Apache webserver on Linux.

9.2. Conclusion/Reflection

This activity is primarily a guide for hosting web applications. What other tools and/or services one might need in order to design, develop, and maintain a functional web application aside from `apache2`?

For making a functional web application, we would use applications like VSCode, MySQL, and the like to store data
For infrastructure, we would use cloud platforms for modern deployment.
For security, we would use OWASP to make sure we have a secure platform.